

瑞幸咖啡前端社招一面面经

瑞幸咖啡前端社招面经

1. 刚刚说到登录，让你设计登录的话，考虑什么，会用到什么技术？
2. 浏览器要ajax，怎么避免明文传输
3. 这个token前端是如何携带上的？【axios请求拦截器】
4. token存哪里？
5. 你现在做的基于Vue？
6. 路由上做是否登录的判断进入不同页面，该如何做？
7. div的文本溢出如何解决？【固定的CSS，或者JS做截取】水平垂直方案有哪些？
8. absolute定位，如果父亲不是relative，那么是参考谁？
9. css中除了PX，还有什么单位？
10. 你觉得问一个问题有很多发散，那我们开始发散，那么浏览器如何让字体是8px？常规最少是9px
11. 隐藏一个元素的方式有哪些？
12. 那透明度设置为0和visibility: hidden有啥区别？
13. 文件上传以来input，type=file，那么经常重写样式，input透明度为0，这是可以的，就用到了透明度为0，面试官解释触发情况我们前端页面，还有JS，有哪些基本数据类型如何判断对象是一个数组？
14. JS有时候用到闭包写法，什么是闭包？
15. 闭包会引发什么问题？
16. es5后有箭头函数，和普通函数的区别？
17. URL输入到页面渲染全过程
18. 你之前项目用的vue，那么react接触过吗？
19. vue有v-if和v-show的区别？
20. 那v-for会给key，key的作用是什么？
21. 我觉得vue对你来说太简单了，vue和react有啥区别？
22. 数学题，有两个杯子1个4L一个9L，怎么倒出6L？
23. 现在能等概率生成1-3，整数，如何等概率生成1-9你的项目中的打包部署是？

参考答案：

1. 登录设计考虑与技术

考虑：

- 安全性：
 - HTTPS: 防止中间人攻击，保证数据传输的安全性。
 - 加密: 在客户端和服务端对密码等敏感信息进行加密（如SHA-256或bcrypt）。
 - 令牌: 使用JWT（JSON Web Token）进行身份验证，减少会话劫持风险。
 - 防止暴力破解: 实现登录失败次数限制和CAPTCHA。
- 用户体验：
 - 简单明了: 提供清晰的输入框和错误提示。
 - 社交登录: 支持通过Google、Facebook等社交平台登录。
 - 记住我功能: 通过安全的cookie存储登录状态。

技术：

- 前端: HTML, CSS, JavaScript, Vue.js（或React.js）。
- 后端: Node.js, Express, Django等。
- 数据库: MongoDB, MySQL。
- 身份验证: JWT, OAuth2。
- 加密: bcrypt, SHA-256。

2. 浏览器中的AJAX避免明文传输

使用HTTPS

这是最基础也是最重要的措施。通过使用HTTPS协议，确保数据在传输过程中被加密。配置HTTPS后，所有通过AJAX发送的请求和响应都会被加密，防止数据被窃取或篡改。

```
1 javascript
2 复制代码
3 $.ajax({url: 'https://example.com/api', type: 'GET', success: function(response)
   {console.log(response);
4   }
5 });
```

2. 使用加密算法对敏感数据进行加密

在发送之前使用加密算法对敏感数据进行加密，在服务器端再进行解密。可以使用如AES这样的对称加密算法。

前端加密（示例使用CryptoJS库）

```
1 javascript
2 复制代码
3  // 引入CryptoJS库
4  <script src="https://cdnjs.cloudflare.com/ajax/libs/crypto-js/3.1.9-1/crypto-
    js.js"></script>
5
6  <script>const secretKey = 'your-secret-key';const plainText = 'sensitive data';
7  // 加密数据const encryptedData = CryptoJS.AES.encrypt(plainText,
    secretKey).toString();
8  // 发送加密后的数据
9  $.ajax({url: 'https://example.com/api',type: 'POST',data: { data:
    encryptedData },success: function(response) {console.log(response);
10      }
11  });
12 </script>
```

服务器端解密（示例使用Node.js和CryptoJS库）

```
1 javascript
2 复制代码
3  const CryptoJS = require('crypto-js');
4
5  const secretKey = 'your-secret-key';
6
7  // 解密数据
8  app.post('/api', (req, res) => {const encryptedData = req.body.data;const
    bytes = CryptoJS.AES.decrypt(encryptedData, secretKey);const plainText =
    bytes.toString(CryptoJS.enc.Utf8);
9      console.log(plainText); // 输出解密后的数据
10      res.send('Data received and decrypted');
11  });
```

3. 使用Token进行身份验证

在请求头中携带Token进行身份验证，避免在请求体中传输敏感信息。

```
1 javascript
```

```
2 复制代码
3  // 设置请求头携带Token
4  $.ajax({url: 'https://example.com/api',type: 'GET',headers: {'Authorization':
    'Bearer your-token'
5  },success: function(response) {console.log(response);
6  }
7  });
```

4. 数据签名

对发送的数据进行签名，确保数据的完整性和来源的可信性。签名可以使用HMAC等算法。

前端签名（示例使用CryptoJS库）

```
1 javascript
2 复制代码
3  <script>const secretKey = 'your-secret-key';const data = 'sensitive data';
4  // 计算数据签名const signature = CryptoJS.HmacSHA256(data, secretKey).toString();
5  // 发送数据和签名
6  $.ajax({url: 'https://example.com/api',type: 'POST',data: { data: data,
    signature: signature },success: function(response) {console.log(response);
7  }
8  });
9  </script>
```

服务器端验证签名（示例使用Node.js和CryptoJS库）

```
1 javascript
2 复制代码
3  const CryptoJS = require('crypto-js');
4
5  const secretKey = 'your-secret-key';
6  app.post('/api', (req, res) => {const data = req.body.data;const signature =
    req.body.signature;
7  // 计算服务器端的签名const serverSignature = CryptoJS.HmacSHA256(data,
    secretKey).toString();
8  // 验证签名if (signature === serverSignature) {console.log('签名验证通过');
9  res.send('Data received and verified');
10 } else {console.log('签名验证失败');
11 res.status(400).send('Invalid signature');
12 }
13 });
```

3. 前端携带token的方式 (axios请求拦截器)

```
1 // 设置请求拦截器
2 axios.interceptors.request.use(config => {
3   const token = localStorage.getItem('token');
4   if (token) {
5     config.headers['Authorization'] = `Bearer ${token}`;
6   }
7   return config;
8 }, error => {
9   return Promise.reject(error);
10 });
```

4. token存储位置

- LocalStorage: 持久化存储, 但存在XSS风险。
- SessionStorage: 会话级存储, 相对更安全, 但关闭浏览器后会失效。
- HttpOnly Cookie: 由服务器设置的HttpOnly Cookie, 更安全, 但需要后端支持。

5. 基于Vue

是的, 目前的项目是基于Vue.js进行开发的。

6. 路由上的登录判断

```
1 // 使用Vue Router进行路由守卫
2 router.beforeEach((to, from, next) => {
3   const isAuthenticated = !!localStorage.getItem('token');
4   if (to.matched.some(record => record.meta.requiresAuth)) {
5     if (!isAuthenticated) {
6       next({ path: '/login' });
7     } else {
8       next();
9     }
10  } else {
11    next();
12  }
13 });
```

7. 文本溢出解决方案

CSS解决方案:

- 单行文本溢出:

```
1 .text-overflow {
2   white-space: nowrap;
3   overflow: hidden;
4   text-overflow: ellipsis;
5 }
```

- 多行文本溢出:

```
1 .multi-line-text-overflow {
2   display: -webkit-box;
3   -webkit-box-orient: vertical;
4   -webkit-line-clamp: 3; /* 行数 */
5   overflow: hidden;
6 }
```

JS解决方案:

- 截取字符串并添加省略号:

```
1 function truncateText(text, length) {
2   if (text.length > length) {
3     return text.substring(0, length) + '...';
4   }
5   return text;
6 }
```

8. absolute定位参考

- 如果父元素没有设置 `position: relative;` 或其他定位属性, `absolute` 会参考最近的定位祖先元素, 如果没有则参考 `<html>` 根元素。

9. CSS单位

- 相对单位: %, em, rem, vh, vw
- 绝对单位: px, pt, cm, mm, in

10. 浏览器设置8px字体

- 尽管一些浏览器默认最小字体是9px, 但可以通过CSS强制设置:

```
1 body {
2   font-size: 8px;
3 }
```

11. 隐藏元素的方式

- `display: none;`
- `visibility: hidden;`
- `opacity: 0;`
- `position: absolute; left: -9999px;`
- `z-index: -1;`

12. 透明度设置为0和visibility区别

- `opacity: 0;`: 元素仍占据布局空间，只是不可见。
- `visibility: hidden;`: 元素不可见但仍占据布局空间。

13. 判断数据类型与数组

基本数据类型:

- `String`, `Number`, `Boolean`, `Null`, `Undefined`, `Symbol`, `BigInt`

判断数组:

- `Array.isArray(obj)`

14. 闭包

闭包是指在函数内部创建的函数可以访问其外部函数的作用域。

```
1 function outer() {
2   let count = 0;
3   return function inner() {
4     count++;
5     return count;
6   }
7 }
8 const increment = outer();
9 console.log(increment()); // 1
10 console.log(increment()); // 2
```

15. 闭包引发的问题

- 内存泄漏: 闭包引用的变量不会被垃圾回收机制回收，可能导致内存泄漏。
- 调试困难: 闭包会增加代码的复杂性，调试变得困难。

16. 箭头函数与普通函数的区别

- 没有自己的 `this`: 箭头函数继承自上下文。

- 没有 `arguments` 对象: 箭头函数没有 `arguments`，可以使用剩余参数。
- 不能作为构造函数: 箭头函数不能使用 `new` 关键字。

17. URL输入到页面渲染全过程

- DNS解析: 将域名解析为IP地址。
- TCP连接: 建立TCP三次握手连接。
- 发送HTTP请求: 浏览器向服务器发送HTTP请求。
- 服务器响应: 服务器返回HTML文档。
- 浏览器解析: 浏览器解析HTML，生成DOM树和CSSOM树，结合生成渲染树。
- 渲染页面: 根据渲染树布局和绘制页面。

18. React的接触

我主要使用Vue.js进行开发，但对React也有所了解，熟悉其基本概念和用法。

19. v-if和v-show的区别

- v-if: 条件渲染，元素在条件为真时才会被渲染或移除。
- v-show: 条件显示，元素始终存在于DOM中，但通过CSS控制显示与隐藏。

20. v-for中的key的作用

key的作用是为了高效地更新渲染列表元素，避免重复渲染，提升性能。

21. Vue与React的区别

- 数据绑定: Vue使用双向数据绑定，React使用单向数据流。
- 模板语法: Vue使用模板语法，React使用JSX。
- 框架与库: Vue是一个框架，React是一个库。

22. 数学题：4L和9L倒出6L

- 4L的杯子装满倒入9L的杯子。
- 4L的杯子再次装满倒入9L的杯子，9L杯子剩下7L。
- 倒掉9L的水，将4L的杯子剩下3L倒入9L杯子。
- 4L的杯子装满倒入9L的杯子，此时9L的杯子有6L。

23. 生成1-9的随机数

```
1 function randomOneToNine() {  
2   return Math.floor(Math.random() * 9) + 1;  
3 }  
4
```


打包部署

使用Webpack进行打包，通过CI/CD工具进行部署，如Jenkins或GitHub Actions。